Sparsity Is Not Speed: An Exhaustive CPU-Latency Analysis of Structured Neural Network Pruning

[Author Name]
[Department]
[Organization]
[City, Country]
[e-mail address]

***Abstract*—Pruned networks are often compared by parameter sparsity or multiply–accumulate (MAC) count, yet deployment cost is paid in latency. We test how well these proxies predict measured CPU latency after structured pruning. For SimpleCNN and ResNet-8 on CIFAR-10 and CIFAR-100, we physically compact all 1,296 policies of a four-unit, six-action layer-wise pruning space and measure batch-1, single-thread latency on an Intel Core i7-1260P under a pre-registered protocol. Parameter reduction correlates weakly with latency gain (Spearman ρ = 0.31–0.55) and MAC reduction strongly (ρ = 0.89–0.98); in 31–39% of policy pairs, the sparser policy is the slower one. Where pruning happens matters: removing 51% of SimpleCNN's parameters from its fully connected layer yields a 1.03× speed-up, whereas a policy that prunes convolution filters and removes 19% of the parameters yields 2.08×. We then estimate the latency headroom that survives a 98% recovered-accuracy constraint for ResNet-8, using a one-epoch recovery proxy validated (ρ = 0.98) and calibrated against five-epoch recovery. The maximum speed-up in this calibrated proxy-feasible region is 1.098×. Accuracy-feasible latency headroom should therefore be measured before expensive hardware-aware optimization is attempted.**

Keywords—structured pruning, CPU inference latency, neural network compression, hardware-aware inference, MACs, accuracy–latency trade-off

I. Introduction

Deep neural networks are increasingly deployed on CPUs in laptops, edge servers and embedded devices, where inference cost is paid in time rather than in parameters [1]. Pruning reduces this cost by removing weights, channels or neurons from a trained network [2], [3], and pruned models are commonly compared by the fraction of parameters removed or by the floating-point operations (FLOPs) or multiply–accumulates (MACs) saved. On a CPU serving interactive or streaming requests, inference often runs at batch size one, where the latency of a single request rather than throughput determines responsiveness.

These counts are convenient, but they are proxies. Unstructured sparsity leaves tensor shapes unchanged, and dense CPU kernels do not skip zeroed weights, so a highly sparse model needs special software or hardware support to run faster [2], such as the sparsity-aware CPU runtime used by Kurtic *et al.* [4]. In convolutional networks, moreover, many parameters sit in fully connected layers while most computation sits in convolutions, so the number of parameters removed says little about where time is saved. Even when computation is physically removed, latency also depends on memory access, parallelism and per-operator work that arithmetic counts do not capture, and recent architecture and compression studies report that reductions in theoretical complexity do not translate proportionally into real-hardware speed-ups [5]–[7].

Structured pruning removes whole filters, channels or neurons and therefore shrinks tensors physically [3], [8]. It is the natural setting in which to ask whether compression proxies track deployment cost, because every structured policy yields a smaller dense network whose latency can be measured directly. Hardware-aware pruning methods already bring latency into their objective, typically through lookup tables or learned cost models [9], [10], but they observe the latency landscape only through the candidates that their search visits, so the headroom that remains once accuracy must be preserved is not characterized before the search is run.

This paper examines four questions on one commodity CPU: whether parameter sparsity reliably predicts inference latency after structured pruning; whether MAC reduction predicts it better; how much latency headroom remains when the pruned network must be retrained and keep 98% of its recovered accuracy; and whether a hardware-aware search should be attempted before such headroom has been verified. We study two CIFAR-scale networks, SimpleCNN and ResNet-8, on CIFAR-10 and CIFAR-100, with a four-unit, six-action layer-wise pruning space that is small enough to enumerate, so that every policy can be physically compacted and timed on an Intel Core i7-1260P laptop CPU. The small space is a deliberate choice: it trades model scale for complete coverage, so that conclusions about cost metrics and latency do not depend on how a search samples the space. We characterize the latency landscape exhaustively and evaluate accuracy-feasible latency headroom using a pre-registered recovery proxy calibrated against five-epoch recovery. All protocols and decision rules were committed to version control before the corresponding data were collected.

This work makes three contributions. First, it provides an exhaustive latency characterization of all 1,296 physically compacted structured pruning policies in each of four network–dataset settings, showing that parameter sparsity and measured CPU latency can diverge substantially. Second, it compares parameter reduction, MAC reduction and measured latency across these policies, finding a weak parameter–latency correlation, a strong MAC–latency correlation, and frequent cases in which a more parameter-sparse policy is slower. Third, it reports a pre-registered recovery and feasibility analysis indicating that, for ResNet-8 on this platform, the region associated with 98% five-epoch accuracy retention remains below the pre-registered 1.10× speed-up criterion, which can leave too little headroom to justify an expensive hardware-aware search. The study proposes no new pruning algorithm, and its conclusions are specific to the measured CPU and networks. Section II reviews related work, Section III describes the methodology and experimental setup, Section IV reports the results, and Sections V and VI discuss and conclude.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

II. Related Work

Pruning research spans the unstructured removal of individual weights and the structured removal of filters, channels, attention heads or layers, and recent surveys organize the field by granularity, importance criterion and the schedule of pruning and retraining [2], [3]. Unstructured sparsity reaches high compression with little loss of accuracy but turns into speed only with special software or hardware support [2]; Kurtic *et al.*, for example, report CPU speed-ups for pruned language models using a sparsity-aware runtime [4]. Structured pruning instead yields a smaller dense network that standard libraries execute without modification. CHEX explores channel configurations by repeatedly pruning and regrowing channels during training and re-allocates the number of channels across layers [11], and Li *et al.* found that randomly sampled channel configurations, trained under a common protocol, can perform comparably to sophisticated importance criteria [12], which suggests that the resulting architecture matters more than the selection rule. Modern architectures complicate removal, because channels joined by residual additions or concatenations must be removed together. DepGraph models these couplings explicitly and groups the parameters that have to be pruned jointly [8], and OTOv2 automates such dependency-aware structured pruning for general networks [13]. In language models, structured pruning of attention heads and hidden dimensions has likewise produced measured inference speed-ups [14].

Because layers differ in size and sensitivity, the fraction removed from each layer is itself a design decision. Rule-based allocations derive per-layer budgets from statistics of weights and activations; OWL, for example, sets non-uniform layer-wise sparsity for large language models from the share of outlier activations in each layer [15]. Such rules are inexpensive but do not consult the target hardware. Learning-based methods search the allocation instead, and structured-pruning surveys group them into regularization, reinforcement-learning and architecture-search approaches [3]. GNN-RL represents the network as a graph and uses reinforcement learning to find a compression policy under a FLOPs constraint [16], and DECORE assigns a reinforcement-learning agent to every channel to learn which channels to keep [17]. In these formulations the reward or constraint is typically a count of parameters or FLOPs, or a latency value supplied by a model of the hardware, so the fidelity of that cost signal directly shapes the policies that the search can find.

Hardware-aware methods bring the target platform into the loop. HALP casts structural pruning as a knapsack problem that maximizes accumulated importance under a latency budget estimated from a per-layer latency lookup table and reports throughput gains of 1.60× and 1.90× for ResNet-50 and ResNet-101 on ImageNet, measured mainly on a GPU and also on other platforms, including an Intel CPU [9]. SMCP likewise constrains latency through a layer-wise lookup table, solves the allocation as a multiple-choice knapsack problem and determines layer-wise pruning ratios automatically [10]. Hardware-aware architecture search pursues the same goal for platforms ranging from microcontrollers to CPUs, GPUs and accelerators [18], and often relies on a learned latency predictor; Akhauri and Abdelfattah show that such predictors transfer across devices with significant variability and improve them for few-measurement settings [19]. Lookup tables and predictors make large searches affordable, but their estimation error enters the objective directly, and when candidate policies differ by only a few percent in latency, a small error can reorder them. Measured latency is itself sensitive to the measurement protocol: batch size, thread count, warm-up and the operator library all change absolute timings, so comparisons are meaningful only under fixed and reported conditions.

These efforts rest on the observation that arithmetic counts are indirect measures of speed. Chen *et al.* show that many networks designed to reduce FLOPs achieve low FLOPS on the hardware, because frequent memory access limits throughput, so fewer FLOPs need not mean lower latency, an effect they report on GPU, CPU and ARM processors [5]. DepthShrinker targets the real-hardware efficiency of compact networks, whose compact operators reduce theoretical complexity but reach low hardware utilization [6], and latency-aware dynamic networks replace FLOPs with a latency prediction model because theoretical efficiency can hardly be translated into realistic speed-ups on server and edge GPUs [7]. Dehghani *et al.* argue more generally that parameters, FLOPs and speed are often assumed to be correlated, so that only some of them are reported, although a model with few FLOPs may not be fast [20]. Comparisons are further complicated because pruning methods are evaluated with different architectures, tasks and experimental settings [2], [12]. Parameter count adds a further distortion in convolutional networks, whose parameters often reside in fully connected layers that contribute little computation, so the layers that dominate model size and those that dominate run time need not coincide. At batch size one, part of each inference may also be spent in work that does not shrink with channel width, which limits how much of a MAC reduction appears as a latency reduction.

Existing methods therefore often optimize or report sparsity, FLOPs or MACs, and hardware-aware methods typically measure latency for the candidates that their search visits or estimate it with a model. Fewer studies characterize an entire structured pruning space with measured CPU latency while also testing how much of that latency survives when the pruned networks must recover a fixed share of their accuracy. For a small, finite policy space such a characterization is feasible, and it answers a question that precedes the choice of any search method: whether an accuracy-feasible region with meaningful latency savings exists at all. It also separates two causes of a weak search outcome, an optimizer that fails to find good policies and a problem that offers none. On the accuracy side, the role of retraining is itself debated [2], and recovered accuracy depends on the recovery budget, so proxy quality matters as well. When training every candidate is too expensive, a cheaper proxy decides which candidates count as good; benchmark studies of zero-cost proxies show that their agreement with trained accuracy varies widely across search spaces and tasks [21], and a proxy that ranks policies poorly near a feasibility boundary can mislead a latency-aware search as much as an inaccurate latency model. Rather than measuring or estimating latency only for the candidates that a search visits, this paper measures every policy of a finite structured space directly and pairs the latency table with an accuracy proxy validated and calibrated against five-epoch recovery. The resulting characterization covers two CIFAR-scale networks on one CPU; it does not claim that the findings transfer to other hardware, larger models or other accuracy tolerances.

III. Methodology and Experimental Setup

Fig. 1 summarizes the methodology. Both branches operate on the same structured policy space: branch A characterizes latency and compares cost metrics with it, and branch B validates and calibrates a recovery proxy. A pre-registered latency-room check combines both and decides whether a hardware-aware search is warranted.

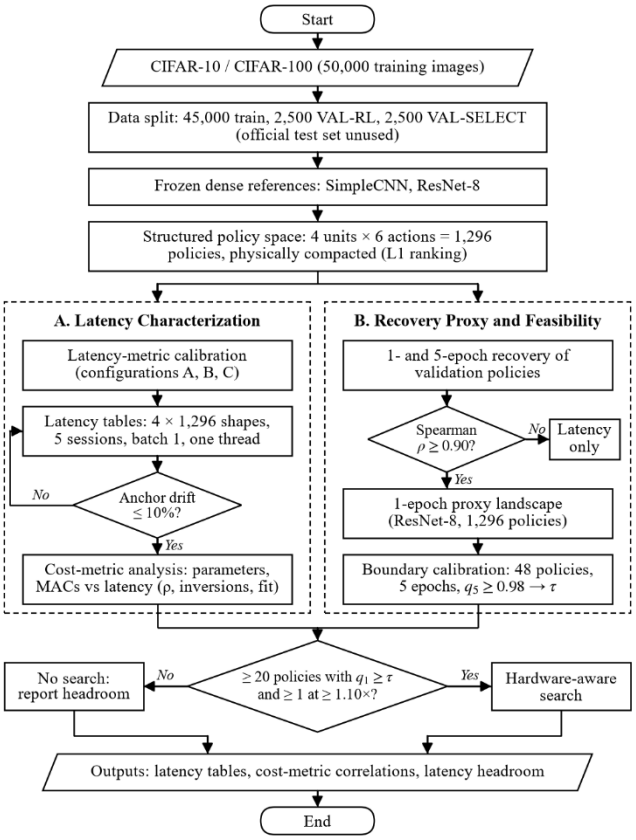


Fig. 1. Overall methodology. All decision rules were fixed before the corresponding data were collected.

A. Networks and Data

We use CIFAR-10 and CIFAR-100. The 50,000 training images are split once into 45,000 for training and 5,000 for validation, and the validation images are divided into two stratified halves of 2,500: VAL-RL, used for every accuracy reported here, and VAL-SELECT, which is held back. The official test set is not used. SimpleCNN has three 3×3 convolutions with 32, 64 and 128 filters, a 256-unit hidden layer and a linear classifier. ResNet-8 is a CIFAR residual network with a 16-channel stem and three basic blocks of 16, 32 and 64 channels. The CIFAR-10 variants contain 620,362 and 78,042 parameters, respectively, and the CIFAR-100 variants 643,492 and 83,892. Each pruned model starts from a frozen, hash-verified dense reference trained beforehand with a fixed recipe, and all code uses PyTorch 2.7.1 [22].

B. Structured Action Space

Each network has four pruning units, and each unit takes one of six actions that remove $r \in \{0, 10, 20, 30, 40, 60\}$% of its channels. A unit with C channels keeps

$$k = \max(1, \text{round}((1 - r)C)) \tag{1}$$

channels with the largest L1 norm [3], scored on the dense model so that the result does not depend on the order of the units. For SimpleCNN the units are the output filters of the three convolutions and the hidden neurons of the first fully connected layer; the following layer loses the matching inputs, including the 16 flattened columns of each removed channel, and the classifier is never pruned. For ResNet-8 the first unit is the stage-1 residual width: one channel set is removed from the stem convolution and its batch normalization, from the outputs of the second convolution of block 1, and from the inputs of every layer that reads this residual stream, including the identity and projection shortcuts [8]. These channels are ranked by the summed L1 norm of the two filters that write them. The other three units are the internal channels of the three blocks.

All 1,296 action vectors give distinct physical shapes in both networks. Slimmed models and the equivalent masked models give the same logits (maximum difference 9.5×10^{-6}), parameter and MAC counts equal closed-form values for all 1,296 policies, and the FLOPs reported by PyTorch's FlopCounter equal twice the MACs.

C. Latency Measurement

All timing runs on a laptop with an Intel Core i7-1260P (four performance and eight efficiency cores), Windows 11 and the PyTorch 2.7.1 CPU build with oneDNN 3.7.1, on AC power. The primary configuration uses batch size 1, one intra-op and one inter-op thread, and a process pinned to one hyper-thread of a performance core before PyTorch is imported. Each model is timed in evaluation and inference mode on a single preallocated float32 input, and model construction is never timed. Every shape receives 50 warm-up and 300 timed passes, measured with a nanosecond counter.

A latency table covers all 1,296 shapes of one network and is measured in five fresh-process sessions, each in a seeded random order. A block of dense-model passes after every 25 shapes monitors drift, and a session whose anchors drift by more than 10% is repeated. The latency L of a shape is the median over sessions of its per-session median, and its gain is

$$g_L = 1 - L / L_{\text{dense}} \tag{2}$$

where L_{dense} is the unpruned shape of the same table.

D. Recovery Protocol and Pre-Registered Rules

Removing channels without retraining is destructive: removing 10% of every unit of ResNet-8 leaves 21% validation accuracy. Every evaluated policy is therefore recovered with one fixed protocol, identical for every policy: Adam with learning rate 10^{-4}, a cosine schedule over the recovery budget, batch size 128, random crop and horizontal flip, no weight decay, and one fixed seed that gives every policy the same minibatch order. We report the retained accuracy

$$q = A / A_{\text{dense}} \tag{3}$$

where A is the VAL-RL accuracy after recovery and A_{dense} that of the dense model after the same recovery. A complete five-epoch recovered-accuracy landscape across all four settings, comprising 5,184 policies, was estimated from measured run times to require about 130 hours of continuous computation on this laptop and was therefore not completed. The exhaustive ResNet-8 recovery landscape uses one-epoch recovery (q_1), while five-epoch recovery (q_5) is reserved for proxy validation and the pre-registered boundary calibration.

Three rules were committed before their data existed. First, the one-epoch proxy is accepted for a network only if the Spearman correlation between one-epoch and five-epoch accuracy on a validation sample is at least 0.90. Second, the proxy threshold τ is the largest q_1 that retains every policy with $q_5 \geq 0.98$ in a stratified calibration sample of 48 policies and leaves at least 20 policies in the full table, minus a fixed margin of 0.005. Third, a hardware-aware search proceeds only if at least 20 policies pass τ and at least one of them is at least 1.10× faster than dense in the latency table.

IV. Results

A. Choice of the Latency Metric

Before building the tables, we timed six SimpleCNN calibration models in three configurations: batch 1 with one thread (A), batch 1 with four threads (B), and batch 128 with four threads (C). A configuration was usable if the latency difference between the dense and the most slimmed model was at least five times the between-session spread, at least 10% of the dense latency, and positive in every session; the first usable configuration in the order A, C, B became the primary metric. Configuration A gave a signal-to-noise ratio of 12.3, C 9.6 and B 4.3, so batch-1, single-thread latency is the primary metric, with a between-session coefficient of variation of 2.1% for the dense model (Table I). Already in this small set, location matters: the fc-heavy model removes 51% of the parameters but only 3% of the MACs and is 1.03× faster, whereas the conv-heavy model removes 63% of the parameters and 65% of the MACs and is 1.47× faster.

TABLE I. Latency Calibration on CIFAR-10 SimpleCNN

Model	Params ↓	MACs ↓	Median ms	Speedup
Dense	—	—	0.680	1.00×
Light	19.1%	17.5%	0.609	1.12×
Moderate	50.7%	49.5%	0.454	1.50×
Aggressive	83.9%	81.7%	0.261	2.60×
FC-heavy	51.1%	2.9%	0.657	1.03×
Conv-heavy	62.6%	65.2%	0.465	1.47×

Light, moderate and aggressive remove 10%, 30% and 60% of every unit; FC-heavy removes 60% of fc1; conv-heavy removes 60% of conv2 and conv3. Configuration A, five sessions.

B. Exhaustive Latency Tables

Table II summarizes the four tables. SimpleCNN shapes span 0.30–0.79 ms and ResNet-8 shapes 0.46–0.77 ms. Across sessions, the median per-shape coefficient of variation was 1.2–1.6%, and the rank correlation of latency gains between any two sessions was at least 0.96. The absolute dense latency differed by 16% between the calibration day and the table measurements while relative gains were stable, so only gains relative to a dense model measured in the same sessions are interpreted. The CIFAR-100 tables reproduce the CIFAR-10 findings: the maximum speed-ups differ by at most 0.05×, the rank correlations by at most 0.012 and the inversion rates by at most 0.4 points, since the two datasets change only the width of the classifier.

Parameter reduction is a weak predictor of latency gain (Fig. 2a), whereas MAC reduction predicts it well (Fig. 2b). Across all pairs of policies with different parameter reductions, the sparser policy is the slower one in 39% of SimpleCNN pairs and 31% of ResNet-8 pairs; for MAC reduction the inversion rate drops to 14% and 5–6%. The SimpleCNN policy [5, 5, 1, 0] removes only 18.7% of the parameters but 69.3% of the MACs and runs 2.08× faster, while removing 60% of the fully connected layer, which holds most parameters but few MACs, removes 51% of the parameters for about a 1.03× speed-up. A linear fit of latency on MACs has a positive intercept of 0.26 ms for SimpleCNN ($R^2 = 0.81$) and 0.33 ms for ResNet-8 ($R^2 = 0.97$). The positive intercept suggests a substantial latency component that is not removed by channel pruning, which limits the attainable speed-up.

TABLE II. Exhaustive Latency Tables (1,296 Policies Each)

Network	Max speedup	ρ_P	ρ_M	Sparser, slower
SimpleCNN, C10	2.61×	0.32	0.89	39.0%
SimpleCNN, C100	2.59×	0.31	0.89	39.4%
ResNet-8, C10	1.67×	0.55	0.98	30.6%
ResNet-8, C100	1.72×	0.54	0.98	30.8%

ρ_P, ρ_M: Spearman correlation of parameter and MAC reduction with latency gain. Last column: share of policy pairs in which the policy with the larger parameter reduction has the higher latency.

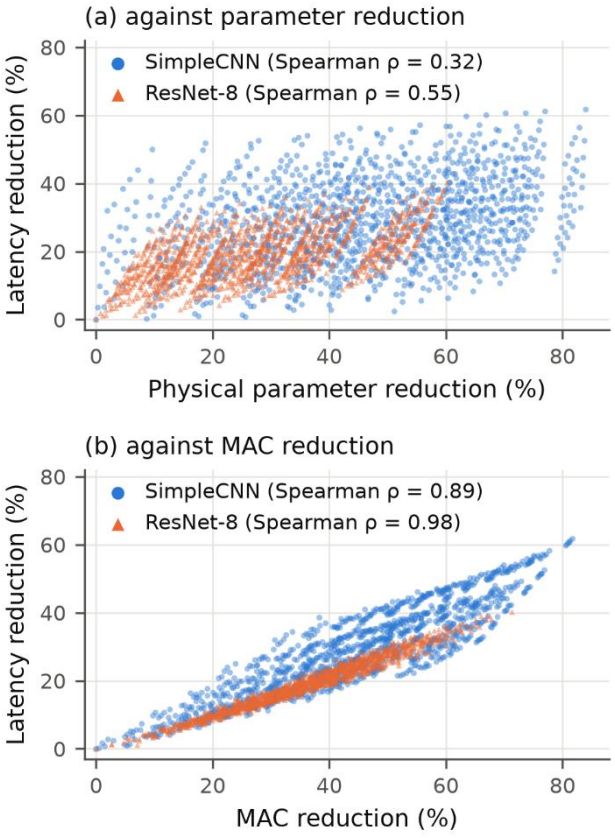


Fig. 2. Latency reduction of all 1,296 physically compacted policies of CIFAR-10 SimpleCNN and ResNet-8 against (a) physical parameter reduction and (b) MAC reduction (batch 1, one thread, i7-1260P).

C. Recovery and Proxy Validity

Five epochs of recovery restore most of the accuracy lost to channel removal: ResNet-8 with 10% removed from every unit improves from 20.96% to 76.84% VAL-RL accuracy, and the conv-heavy SimpleCNN from 25.84% to 73.36%. The one-epoch proxy was validated on 156 SimpleCNN policies (126 lightly pruned pilot policies and 30 random ones) and on 31 ResNet-8 policies (the dense policy and 30 random ones), each recovered for one and for five epochs (Fig. 3, Table III). The proxy underestimates the five-epoch accuracy in every pair. For ResNet-8 it preserves the ranking and was accepted. For SimpleCNN the overall ρ of 0.844 is below the pre-registered 0.90: among the lightly pruned pilot policies, whose five-epoch accuracies lie within a few tenths of a point of each other, the one-epoch ranking is unreliable ($\rho = 0.77$), although

it is accurate across the random policies ($\rho = 0.97$). SimpleCNN's one-epoch proxy therefore failed the pre-registered acceptance threshold and was not used to build an exhaustive proxy accuracy landscape; SimpleCNN contributes only its latency evidence.

TABLE III. VALIDATION OF THE ONE-EPOCH RECOVERY PROXY

Metric	SimpleCNN	ResNet-8
Policy pairs	156	31
Spearman ρ (A_1, A_5)	0.844	0.980
Pearson r	0.973	0.990
Mean bias $A_1 - A_5$ (pp)	−1.29	−5.68
Top-10 overlap	3	10
Decision ($\rho \geq 0.90$)	Rejected	Accepted

A_1, A_5: VAL-RL accuracy after one and five recovery epochs (CIFAR-10).

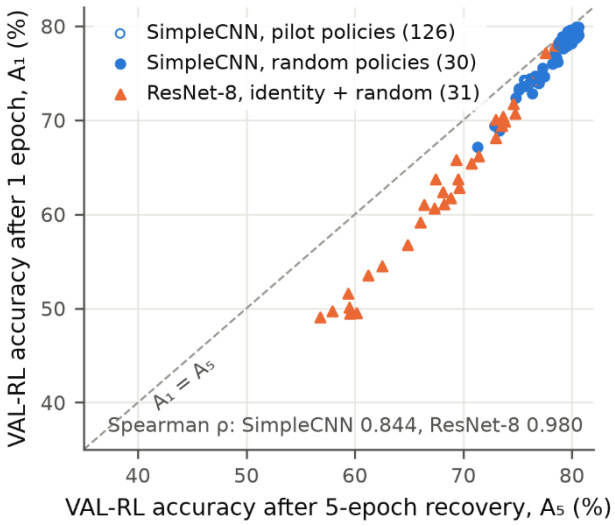


Fig. 3. VAL-RL accuracy after the one-epoch proxy recovery against the five-epoch recovery for the validation policies of both networks. The dashed line marks equality.

TABLE IV. RESNET-8 PROXY-CALIBRATED LATENCY ROOM AT 98% RETENTION

Quantity	Value
Calibration policies (five epochs)	48
Calibration policies with $q_5 \geq 0.98$	14
Proxy threshold τ	0.960
Recall / precision at τ (calibration)	1.00 / 0.875
Proxy-feasible policies, $q_1 \geq \tau$ (of 1,296)	52
Latency of proxy-feasible policies	0.702–0.770 ms
Best proxy-feasible speed-up (rule: ≥ 1.10×)	1.098×
Best five-epoch-feasible calibration policy	1.089×

Proxy-feasible policies are candidates identified by the calibrated one-epoch proxy; only the 48 calibration policies have five-epoch accuracy.

D. Latency Room under an Accuracy Constraint

For ResNet-8, the one-epoch proxy was computed for all 1,296 policies. The stratified calibration sample of 48 policies, eight per bin of q_1, contained 14 policies with $q_5 \geq 0.98$ (Table IV). The pre-registered rule gave $\tau = 0.960$, with recall 1.00 and precision 0.875 on the sample. The calibrated proxy identifies 52 candidate policies associated with the five-epoch feasibility region. The fastest of them is 1.098× faster than dense, and the fastest truly feasible calibration policy is 1.089× faster; every faster calibration policy, up to 1.22×, has a five-epoch retention below 0.98 (Fig. 4). The pre-registered 1.10× requirement was therefore not met, and no hardware-aware search was started. The gap between 1.098× and 1.10× is not scientifically meaningful in itself; the stop is a pre-registered protocol decision, and the substantive finding is that the region identified by the calibrated proxy offers speed-ups of about 10% at most.

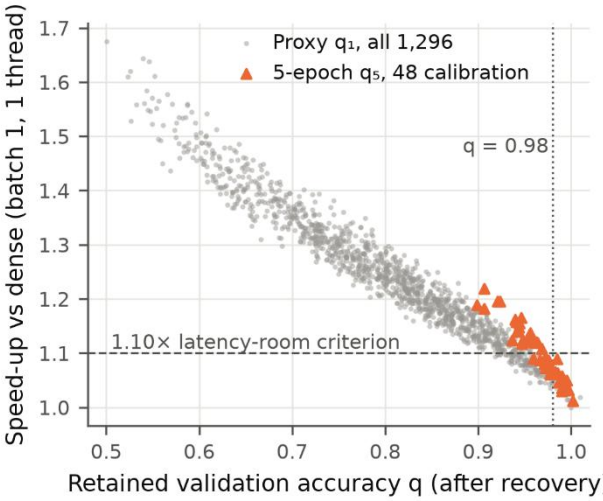


Fig. 4. ResNet-8 speed-up versus retained accuracy. Grey points show one-epoch proxy values for all 1,296 policies, while orange points show five-epoch values for the 48-policy calibration set. The proxy-feasible set reaches at most 1.098× speed-up, and the best truly feasible calibration policy reaches 1.089×.

V. DISCUSSION

A. What the Cost Metric Should Measure

On this CPU, a cost based on parameter count would favor pruning the fully connected layer of SimpleCNN, which holds 85% of its parameters but 5% of its MACs; such policies save memory but little time. MAC reduction ranks policies almost as measured latency does (ρ up to 0.98) and is a sound, hardware-independent first approximation. Measured latency captures shape-specific and non-MAC effects that arithmetic counts miss, and it remains the deployment-grounded metric. For a finite policy space an exhaustive latency table is inexpensive: the four tables took about 2.5 hours and remove timing noise from any downstream objective.

For pruning studies that target CPU deployment, these results suggest reporting measured latency together with the conditions that produced it, namely batch size, thread count, core affinity, operator library and number of sessions, and expressing gains relative to a dense model measured in the same sessions, because absolute latency shifted by 16% between days while relative gains remained stable. MAC counts remain useful as a hardware-independent screen, but their agreement with measurement on the target device should be checked before they serve as an optimization objective [20].

B. Why ResNet-8 Leaves Little Room

ResNet-8 is compact, and its residual stages are sensitive to channel removal. Its latency–MAC fit has a positive intercept of 0.33 ms relative to a dense latency of about 0.77 ms, consistent with a substantial latency component that is not eliminated by channel pruning. The validated proxy indicates that policies compatible with 98% five-epoch retention are concentrated in the lightly pruned region, where MAC reductions reach at most 18% among the proxy-feasible policies and latency falls by at most 9%. The evidence therefore points to the accuracy constraint, rather than search quality alone, as the principal limiter of latency headroom in

this setting: an exhaustive enumeration of the proxy table finds no proxy-feasible policy at 1.10×, although the threshold retained every truly feasible calibration policy.

C. Implications for Hardware-Aware Optimization

The accuracy-feasible latency headroom of a pruning problem can be measured before an optimizer is built. Here an exhaustive latency table was combined with an accuracy proxy validated against five-epoch recovery and a pre-registered feasibility check. Under the calibrated proxy and five-epoch boundary validation, the accuracy-feasible region available to a latency-aware objective, whether pursued by reinforcement learning, evolutionary search or a greedy method, was narrow and remained below the pre-registered 1.10× criterion. Larger networks, a looser accuracy tolerance, longer recovery or batched inference, where configuration C reached a 2.76× speed-up in calibration, are settings in which hardware-aware optimization may have more room to matter.

D. Limitations

All latencies come from one laptop CPU (Intel Core i7-1260P, Windows 11, PyTorch CPU backend with oneDNN), with batch size one and a single thread as the primary metric; absolute latency shifted by 16% between days, so only relative gains are interpreted, and no GPU or energy measurements were made. The study covers two CIFAR-scale networks, SimpleCNN and ResNet-8. All accuracies are validation accuracies from a single recovery seed, and the test set was not used. No exhaustive five-epoch accuracy landscape was computed: the ResNet-8 analysis rests on a validated one-epoch proxy and a 48-policy calibration, and the SimpleCNN proxy was rejected. Because the pre-registered latency-room criterion was not met, no reinforcement-learning or other hardware-aware optimization was run. The 0.98 tolerance and the 1.10× threshold are conventions fixed in advance.

VI. Conclusion

Exhaustive measurement of 5,184 physically compacted structured pruning policies on one CPU shows that parameter sparsity is not a reliable deployment-cost metric, while MAC count is substantially better, with rank correlations of 0.89–0.98 against measured latency gain. Measured latency nonetheless remains the deployment-grounded metric. For ResNet-8, the validated proxy and five-epoch calibration indicate less than 10% latency headroom in the region associated with 98% recovered-accuracy retention. Accuracy-feasible latency headroom should therefore be established before expensive hardware-aware optimization is attempted. Extending the exhaustive approach to larger networks, other processors and batched inference is the natural next step.

References

[1] G. Menghani, "Efficient deep learning: A survey on making deep learning models smaller, faster, and better," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–37, 2023.

[2] H. Cheng, M. Zhang, and J. Q. Shi, "A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 10558–10578, Dec. 2024.

[3] Y. He and L. Xiao, "Structured pruning for deep convolutional neural networks: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 5, pp. 2900–2919, May 2024.

[4] E. Kurtic *et al.*, "The Optimal BERT Surgeon: Scalable and accurate second-order pruning for large language models," in *Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP)*, 2022, pp. 4163–4181.

[5] J. Chen *et al.*, "Run, don't walk: Chasing higher FLOPS for faster neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 12021–12031.

[6] Y. Fu *et al.*, "DepthShrinker: A new compression paradigm towards boosting real-hardware efficiency of compact neural networks," in *Proc. 39th Int. Conf. Mach. Learn. (ICML)*, PMLR vol. 162, 2022, pp. 6849–6862.

[7] Y. Han *et al.*, "Latency-aware spatial-wise dynamic networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 36845–36857.

[8] G. Fang, X. Ma, M. Song, M. B. Mi, and X. Wang, "DepGraph: Towards any structural pruning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 16091–16101.

[9] M. Shen, H. Yin, P. Molchanov, L. Mao, J. Liu, and J. M. Alvarez, "Structural pruning via latency-saliency knapsack," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 12894–12908.

[10] R. Humble, M. Shen, J. A. Latorre, E. Darve, and J. M. Alvarez, "Soft masking for cost-constrained channel pruning," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2022, pp. 641–657.

[11] Z. Hou *et al.*, "CHEX: CHannel EXploration for CNN model compression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 12287–12298.

[12] Y. Li, K. Adamczewski, W. Li, S. Gu, R. Timofte, and L. Van Gool, "Revisiting random channel pruning for neural network compression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 191–201.

[13] T. Chen, L. Liang, T. Ding, Z. Zhu, and I. Zharkov, "OTOv2: Automatic, generic, user-friendly," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[14] M. Xia, Z. Zhong, and D. Chen, "Structured pruning learns compact and accurate models," in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2022, pp. 1513–1528.

[15] L. Yin *et al.*, "Outlier weighed layerwise sparsity (OWL): A missing secret sauce for pruning LLMs to high sparsity," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, PMLR vol. 235, 2024, pp. 57101–57115.

[16] S. Yu, A. Mazaheri, and A. Jannesari, "Topology-aware network pruning using multi-stage graph embedding and reinforcement learning," in *Proc. 39th Int. Conf. Mach. Learn. (ICML)*, PMLR vol. 162, 2022, pp. 25656–25667.

[17] M. Alwani, Y. Wang, and V. Madhavan, "DECORE: Deep compression with reinforcement learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 12349–12359.

[18] K. T. Chitty-Venkata and A. K. Somani, "Neural architecture search survey: A hardware perspective," *ACM Comput. Surv.*, vol. 55, no. 4, pp. 1–36, 2022.

[19] Y. Akhauri and M. S. Abdelfattah, "On latency predictors for neural architecture search," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 6, 2024, pp. 512–523.

[20] M. Dehghani, A. Arnab, L. Beyer, A. Vaswani, and Y. Tay, "The efficiency misnomer," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[21] A. Krishnakumar, C. White, A. Zela, R. Tu, M. Safari, and F. Hutter, "NAS-Bench-Suite-Zero: Accelerating research on zero cost proxies," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 35, 2022, pp. 28037–28051.

[22] J. Ansel *et al.*, "PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation," in *Proc. 29th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2024, pp. 929–947.